

文档加载、清洗与分块设计

DOC_ID: DOC-RAG-CHUNK-001

说明：本文档为 RAGLab 项目生成的教学用合成语料，用于分块、检索、重排、引用和评测实验。它不是 LangChain/LangGraph 官方文档，也不应被当作 API 版本说明。文中的稳定标识（DOC_ID、SECTION_ID）用于构建可重复的检索评测集。

1. 文档加载后的统一表示

SECTION_ID: CHUNK-LOAD-01

不同来源的文件应在加载后转换成统一的 Document 结构，至少包含 `page_content` 和 `metadata`。元数据应尽早建立，因为后期补充来源、页码和章节信息会非常困难。

推荐保留：

```
{
  "doc_id": "DOC-RAG-CHUNK-001",
  "section_id": "CHUNK-LOAD-01",
  "source": "03_document_loading_chunking.pdf",
  "page": 1,
  "heading_path": ["文档加载、清洗与分块设计", "文档加载后的统一表示"],
  "language": "zh-CN"
}
```

2. 清洗不等于删除一切格式

SECTION_ID: CHUNK-CLEAN-01

文档清洗应删除重复页眉、页脚、导航和乱码，但应尽量保留标题层级、列表、表格说明和代码块边界。过度清洗会让后续分块失去结构线索。

例如把所有换行都压成一个空格，会使标题与正文无法区分；删除代码中的缩进，可能改变其含义。

3. 固定长度分块

SECTION_ID: CHUNK-FIXED-01

固定长度分块按字符数或 Token 数切分，实现简单，适合作为基线。主要参数是 `chunk_size` 和 `chunk_overlap`。

它的优点是行为稳定、容易批量处理；缺点是可能在句子、列表或代码中间切断语义。

4. 递归字符分块

SECTION_ID: CHUNK-RECUR-01

递归分块按照预设分隔符层级尝试切分，例如先按章节、段落、句子，再退化为字符级。它比纯固定切分更容易保留自然边界，因此常被作为通用基线。

但是，递归分块仍然不理解文档语义。若章节很长，最终仍会按照长度被拆开。

5. 结构化分块

SECTION_ID: CHUNK-STRUCT-01

Markdown、HTML 和规范技术文档通常具有标题层级。结构化分块先识别标题，再在章节内部控制长度，并将标题路径写入 metadata。

这样用户询问“Checkpoint 与 Store 的区别”时，检索块即使正文没有重复完整标题，也能利用 `heading_path` 提供上下文。

6. 父子文档检索

SECTION_ID: CHUNK-PARENT-01

父子文档策略使用较小的子块进行检索，再返回更大的父块作为生成上下文。它试图同时获得：

- 小块检索的精确性；
- 大块回答的完整性。

例如用 250 Token 的子块建立向量索引，命中后返回所属的 1200 Token 父章节。父子关系必须通过稳定 ID 保存，否则索引更新后难以定位。

7. 分块实验应控制变量

SECTION_ID: CHUNK-EXP-01

比较分块策略时，应固定 Embedding 模型、检索方法、Top-K 和评测集，只改变分块参数。否则无法判断提升来自哪里。

建议第一轮实验：

实验	chunk_size	overlap	结构
A	300	30	固定长度
B	600	80	递归
C	1000	120	递归
D	章节内 600	80	标题结构化
E	子块 250 / 父块 1200	30	父子检索

记录索引块数量、平均块长度、Hit@K、MRR、上下文 Token 和回答忠实性。